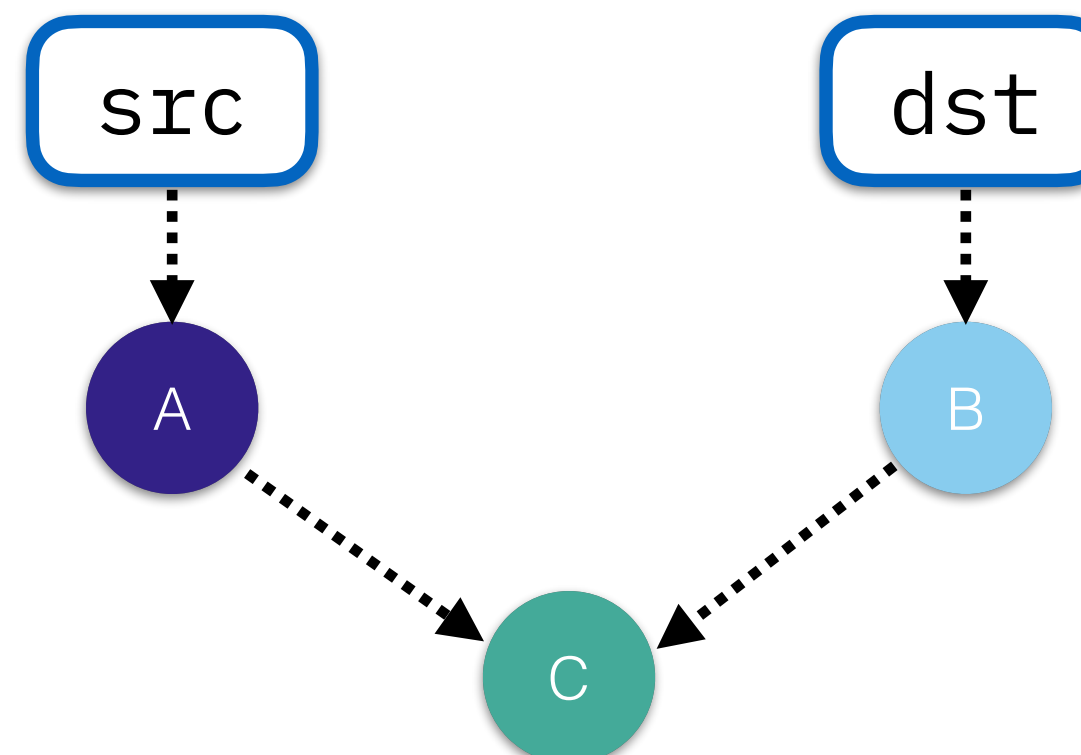


What is BoC?

```
def main(src: cown[Account], dst: cown[Account]) {  
  when(src) { A src.balance += 10; }  
  when(dst) { B dst.frozen = true; }  
  when(src, dst) { C  
    if (!dst.frozen && !src.frozen && src.balance >= 10) {  
      src.balance -= 10; dst.balance += 10;  
    }  
  }  
}
```

Behaviours are executed with respect to the order they were spawned



Why BoC

```
def main(src: cown[Account], dst: cown[Account]) {  
  when(src) { A src.balance += 10; }  
  when(dst) { B dst.frozen = true; }  
  when(src, dst) { C  
    if (!dst.frozen && !src.frozen && src.balance >= 10) {  
      src.balance -= 10; dst.balance += 10;  
    }  
  }  
}
```

- Inspired by actors but decouples isolation from concurrency
- Isolation of resources yields data-race freedom by construction
- Coupling isolation and concurrency makes rendezvous patterns hard to express
- Decoupling them allows us to construct richer ad-hoc coordination patterns